

DESARROLLO DE APLICACIONES MÓVILES

Mano Amiga

Diseño preliminar

Jimena Bazzano · Martina Gutiérrez · Agustina Rabajoli

EL PROYECTO

Una app para coordinar voluntariados



Voluntario

Busca actividades en el mapa y el feed, se inscribe y marca su asistencia.



Coordinador

Publica y etiqueta actividades, acepta inscriptos y valida asistencias.



Administrador

Da de alta organizaciones y coordinadores, y ve las horas de los judiciales.



Voluntario judicial: con ABAC se le ocultan las actividades sensibles (menores, apoyo escolar, hogares) y sus horas se acumulan contra las que dictó el juez.

Características móviles: cámara (QR de asistencia), geolocalización (actividades cercanas) y notificaciones push.

1. Arquitectura de capas

1. ARQUITECTURA DE CAPAS

Cliente React Native, DMZ y red segura

CLIENTE

App React Native con Expo

Corre en el celular del usuario. Es zona no confiable: muestra y pide datos, pero nunca decide la seguridad.

DMZ

Zona expuesta a Internet

La única parte accesible desde afuera. Solo tiene el front-service.

RED SEGURA

Backend protegido

La API y MongoDB. Desde Internet no se llega: solo se entra a través del front-service.



Entre cada capa hay un firewall: **FW 1** separa Internet de la DMZ y **FW 2** separa la DMZ de la red segura.

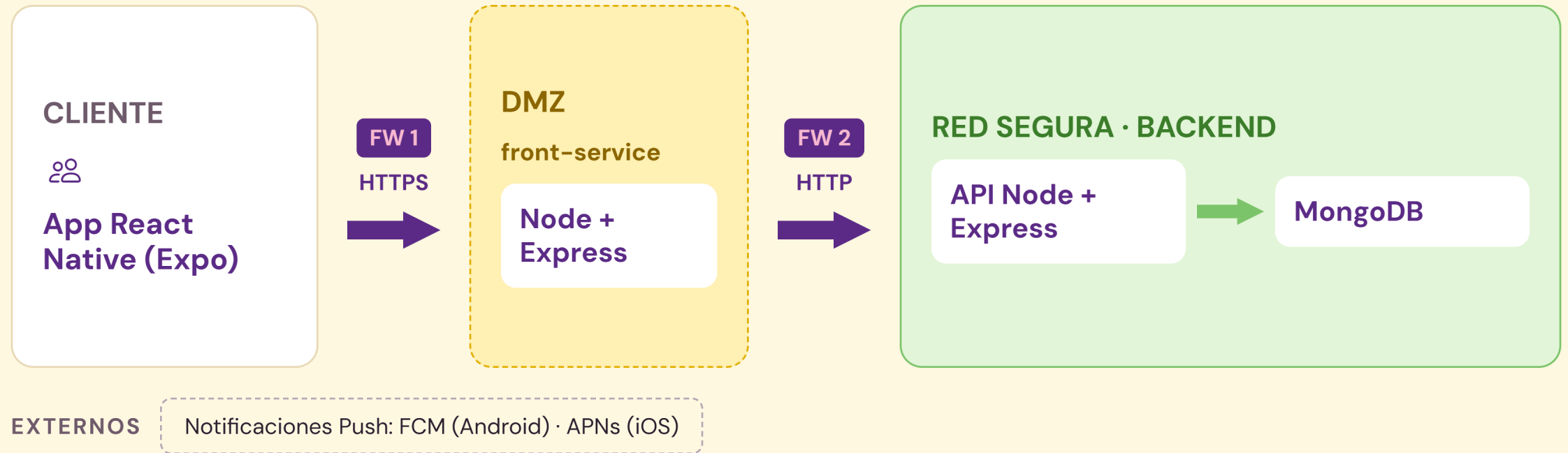
1. ARQUITECTURA DE CAPAS

Componentes principales y su responsabilidad

Capa	Componente	Responsabilidad
Cliente	Pantallas + React Navigation	UI por rol; tabs Inicio, Mapa, Actividades, Perfil
Cliente	AuthContext + cliente axios	Sesión global; agrega el JWT; maneja 401 y errores de red
Cliente	Funciones del dispositivo	Mapa y GPS, cámara para el QR, notificaciones push
DMZ	Front-service (Node + Express)	HTTPS, helmet, rate limit y reenvío a la API
Backend	Controllers	Manejan HTTP: reciben el pedido y responden
Backend	Middlewares	La seguridad: token, permisos y validación
Backend	Services	La lógica de negocio: cupos, QR y horas
Backend	Models	Definen cada tipo de dato y leen y escriben en MongoDB
Backend	MongoDB	Guarda los datos

1. ARQUITECTURA DE CAPAS

Diagrama de arquitectura



2. Comunicación entre capas

2. COMUNICACIÓN ENTRE CAPAS

Protocolos

HTTPS (TLS)

De la app al front-service. Cifra los datos y verifica el servidor con su certificado.

REST + JSON

Recursos por URL, el método HTTP indica la acción y los datos viajan en JSON.

HTTP interno

Del front-service a la API, dentro de la red privada aislada.

Protocolo MongoDB

De la API a la base, siempre con usuario y contraseña.

Push: FCM / APNs

La API envía notificaciones por HTTPS a través de Expo Push.

SSH

Solo para administrar el servidor, desde la IP del equipo y con clave pública.

2. COMUNICACIÓN ENTRE CAPAS

Flujo de una operación completa

1 App

POST /actividades/:id/inscripciones
con el JWT en el header Authorization.

2 Front-service

Atiende el HTTPS, aplica rate limit y la reenvía a api:3000 en la red segura.

3 Middlewares

Verifica el JWT, relee al usuario en la base, controla el rol y valida el :id.

4 Service

Chequea ABAC (judicial × sensible)
y reserva el cupo con una
operación atómica.

5 MongoDB

Guarda la inscripción; el índice único
evita inscribirse dos veces.

6 Respuesta

201 con la inscripción; la app actualiza
la lista y el coordinador recibe un push.

2. COMUNICACIÓN ENTRE CAPAS

Comunicaciones permitidas y puertos

Origen	Destino	Protocolo	Puerto	¿Permitido?
App (Internet)	Front-service · DMZ	HTTPS, REST/JSON, TLS 1.2+	443	Sí
Internet	Front-service · DMZ	HTTP, solo redirige a HTTPS	80	Solo redirect
Front-service · DMZ	API · segura	HTTP interno (proxy)	3000	Sí
API	MongoDB	Protocolo MongoDB + auth	27017	Sí
API	Expo Push → FCM / APNs	HTTPS saliente	443	Sí
App	Google Maps	HTTPS	443	Sí
Equipo (IP fija)	Servidor	SSH con clave pública	22	Restringido
Internet	API o MongoDB	Cualquiera	3000, 27017	No
Front-service · DMZ	MongoDB	Cualquiera	27017	No

Regla por defecto: se deniega todo lo que no está en la tabla

3. API

3. API

Endpoints y autenticación

Método	Ruta y parámetros	Autenticación	Éxito	Errores
POST	/auth/register	Pública	201	400, 409
POST	/auth/login	Pública	200	401, 429
POST	/auth/refresh	Refresh token	200	401
POST	/auth/reauth	Admin	200	401, 403
GET · PATCH	/usuarios/me	JWT	200	400, 401
PATCH	/usuarios/me/password	JWT	204	400, 401
GET	/usuarios/me/accesos	JWT	200	401
POST · DELETE	/admin/organizaciones(/:id)	Admin + reauth	201 · 204	403, 404
POST · DELETE	/admin/coordinadores(/:id)	Admin + reauth	201 · 204	403, 404
GET	/actividades?etiqueta&tipo&lat&lng&radio	JWT + ABAC	200	400, 401
GET	/actividades/:id	JWT + ABAC	200	403, 404
POST	/actividades	Coordinador	201	400, 403
PATCH · DELETE	/actividades/:id	Coordinador de la org	200 · 204	403, 404

3. API

Endpoints y autenticación

Método	Ruta y parámetros	Autenticación	Éxito	Errores
POST	/actividades/:id/inscripciones	Voluntario + ABAC	201	403, 404, 409
GET	/usuarios/me/inscripciones	Voluntario titular	200	401
GET	/actividades/:id/inscripciones	Coordinador de la org	200	403, 404
PATCH	/inscripciones/:id { estado }	Coordinador de la org	200	400, 403, 409
GET	/inscripciones/:id/certificado	Titular o coordinador	200	403, 409
GET	/actividades/:id/qr	Coordinador de la org	200	403, 404
POST	/actividades/:id/asistencias { qr }	Voluntario titular	201	400, 403, 409
PATCH	/asistencias/:id/validar	Coordinador de la org	200	403, 404
GET	/usuarios/me/horas	Voluntario judicial	200	403
GET	/admin/judiciales/horas	Admin	200	403
GET	/mapa/puntos · /campanas · /publicaciones	Pública	200	400
POST	/campanas · /publicaciones	Coordinador de la org	201	400, 403

En naranja: registro de horas · Lo público solo devuelve datos institucionales

3. API

Request/response, códigos y errores

```
POST /api/v1/actividades/64f1.../inscripciones
Authorization: Bearer eyJhbGciOi...

← 201 Created

{ "id": "6512...", "actividadId": "64f1...", "estado":
  "pendiente" }

← 409 Conflict

{ "error": { "code": "CUPO_LLENO", "message": "No quedan
  cupos" } }
```

Código	Cuándo
200 · 201 · 204	OK, creado, sin contenido
400	Datos inválidos
401	Sin token o token vencido
403	Rol o atributo no permitido
404	El recurso no existe
409	Cupo lleno o duplicado
429	Demasiados intentos
500	Error interno, sin detalles

Un middleware central de errores responde siempre con el mismo formato; la app decide qué mostrar según el **code** y, ante un 401, intenta refrescar el token o vuelve al login.

4. Modelo de datos

4. MODELO DE DATOS

Collections de MongoDB y su propósito

Collection	Propósito	Campos clave
usuarios	Cuentas y atributos ABAC	rol · organizacionId · esJudicial · activo
organizaciones	Quién publica	nombre · contacto · convenioOSLA · activa
etiquetas	Define qué es sensible	nombre · sensible (true / false)
actividades	Lo que se publica y va al mapa	organizacionId · etiquetas · ubicacion · cupo
inscripciones	Voluntario ↔ actividad	estado · índice único (actividad, voluntario)
asistencias	Horas verificables	checkIn · checkOut · horas · validadaPor
campanas	Donaciones y acopio	organizacionId · puntos (2dsphere) · fechas
publicaciones	Feed y convocatorias	proyectoId · autorId · tipo · contenido
sesiones	Refresh tokens revocables	userId · tokenHash · expiraEn (TTL)
auditoria	Logins y acciones sensibles	usuario · acción · recurso · fecha · IP

5. Seguridad

5. SEGURIDAD

Autenticación y sesiones con JWT



Contraseñas

Se guardan con bcrypt, nunca en claro ni en logs. El login tiene rate limit y queda auditado.



JWT corto, solo con el id

Firmado con un secreto del servidor, vence a los 15 minutos. No lleva rol ni permisos adentro.



Permisos siempre al día

En cada request se releen rol, organización y estado en la base. Una baja corta el acceso al instante.



Refresh token

Refresh de 7 días, rotativo y revocable.

5. SEGURIDAD

Autorización, validación y secretos

Autorización en 4 niveles

Siempre en el servidor; la app solo oculta.

- 1 **Por rol:** un middleware controla cada ruta
- 2 **Por organización:** cada coordinador, solo lo suyo
- 3 **Por atributos (ABAC):** un judicial no ve lo sensible
- 4 **Admin:** el panel pide reautenticación

✓ Entrada y salida

- Esquema de validación por endpoint
- El rol nunca viene del cliente
- Datos sensibles solo para quien corresponde

🔑 Secretos

Variables de entorno, nunca en el repositorio.



Principio general: mínimo privilegio y denegar por defecto, en la red, en la base y en cada endpoint.

5. SEGURIDAD

Datos sensibles y límites de confianza

Dato sensible	Dónde está	Nivel
Contraseña (hash)	usuarios	Alto
Condición de judicial	usuarios	Alto
Tokens JWT y refresh	celular · sesiones	Alto
Teléfono, nombre y email	usuarios	Medio
Historial de inicios de sesión	auditoria	Medio
Asistencias	asistencias	Bajo
Ubicación de actividades	actividades	Bajo

Celular · App (no confiable)

Límite 1: HTTPS y autenticación

Front-service (DMZ)

Límite 2: autorización por rol y organización

API + MongoDB (red segura)

Límite 3: nunca salen datos personales

Servicios externos (mapas, push)

6. Organización del proyecto

6. ORGANIZACIÓN DEL PROYECTO

Estructura del proyecto React Native

Carpeta	Qué contiene
App.js	Entrada: providers y RootNavigator
src/api/	Instancia axios, interceptores, un módulo por recurso
src/context/	AuthContext: usuario, tokens, login y logout
src/navigation/	AuthStack, AppTabs y stacks según el rol
src/screens/	auth, feed, mapa, actividades, perfil, coordinador
src/components/	ActividadCard, EtiquetaChip, QRScanner...
src/hooks/	useAuth, useActividades, useUbicacion
src/services/	Notificaciones y almacenamiento seguro
src/constants/	Tema y estilos (StyleSheet), URL de la API

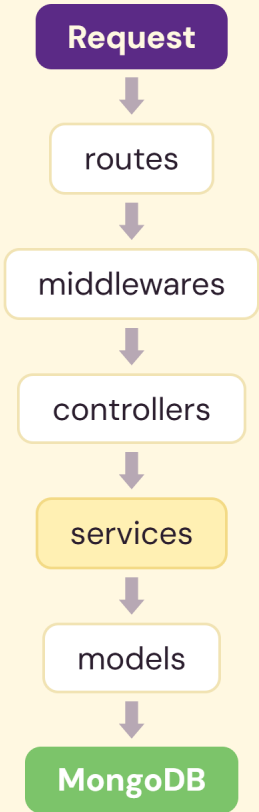
Reglas de separación

- Las pantallas no hacen fetch: usan src/api
- La sesión vive solo en AuthContext
- Los componentes solo reciben props y muestran
- La navegación cambia según el rol

6. ORGANIZACIÓN DEL PROYECTO

Estructura Node/Express por capas

Carpeta	Qué contiene
server.js · app.js	Arranque, conexión a Mongo, middlewares globales
src/config/	Variables de entorno y conexión a la base
src/routes/	Rutas por recurso y sus middlewares
src/middlewares/	auth, authorize, validate, rateLimit, errorHandler
src/controllers/	Leen la request y devuelven la respuesta
src/services/	Negocio: cupos, ABAC, QR, horas
src/models/	Esquemas Mongoose e índices
src/validators/	Esquemas Joi por endpoint
src/utils/	ApiError, logger, helpers
tests/	Pruebas de endpoints y servicios
front-service/	Otro Express chico: helmet, rate limit, proxy



6. ORGANIZACIÓN DEL PROYECTO

Qué corresponde a cada capa

Capa	Le corresponde	No le corresponde
App React Native	Mostrar datos, navegar, guardar la sesión	Reglas de negocio y permisos
Front-service (DMZ)	HTTPS, headers, rate limit, reenvío	Lógica de negocio y datos
Routes + middlewares	URLs, token, rol, ABAC, validación	Lógica de negocio
Controllers	Traducir HTTP: leer la request, armar la respuesta	Consultar la base
Services	Reglas de negocio: cupos, QR, horas	Saber de HTTP
Models + MongoDB	Esquemas, índices y persistencia	Decidir reglas



Módulos por recurso: actividades, inscripciones, asistencias... cada uno con su route, controller, service y model. En la app, un módulo de api por recurso y componentes reutilizables.

7. Despliegue

7. DESPLIEGUE

Dónde se ejecuta y cómo se despliega

Componente	Dónde corre	Puerto	Cómo se despliega
App móvil	Celular (Expo Go en desarrollo)	—	EAS Build genera el APK / AAB
Front-service Node 20	Contenedor en la red DMZ	443 y 80 públicos	Imagen propia + certificado Let's Encrypt
API Node 20	Contenedor en la red segura	3000 interno	Imagen propia + variables de entorno
MongoDB 7	Contenedor + volumen persistente	27017 interno	Imagen oficial, auth activada, mongodump

dmz_net

Solo el front-service. Es el único que publica puertos al host.

secure_net (internal)

front-service, api y mongo. Sin puertos publicados; solo la API sale.

AWS EC2 (capa gratuita)

Linux + Docker. El security group es FW 1. Dominio [dominio].

7. DESPLIEGUE

Firewall, reglas y dependencias

Acción	Tráfico	Puerto
ALLOW	Internet → front-service	443/tcp
ALLOW	Internet → front-service (redirect)	80/tcp
ALLOW	IP del equipo → servidor (SSH)	22/tcp
ALLOW	front-service → api	3000
ALLOW	api → mongo	27017
ALLOW	api → Expo Push / FCM (salida)	443
DENY	Todo lo demás	*

DEPENDENCIAS EXTERNAS

Expo + EAS · build de la app y servicio de push

Firebase Cloud Messaging · entrega de notificaciones

Google Maps SDK · mapa y geolocalización

Let's Encrypt · certificados TLS

npm · express, mongoose, jsonwebtoken, bcrypt, helmet, http-proxy-middleware

Muchas gracias

Jimena Bazzano · Martina Gutiérrez · Agustina Rabajoli